

Mobile Delivery Robots: Mixed Reality-Based Simulation Relying on ROS and Unity 3D

Yuzhou Liu¹ *Student Member, IEEE*, Georg Novotny^{1,2}  *Student Member, IEEE*,
Nikita Smirnov^{1,3}  Walter Morales-Alvarez¹  *Student Member, IEEE*
and Cristina Olaverri-Monreal¹  *Senior Member, IEEE*

Abstract—In the context of Intelligent Transportation Systems and the delivery of goods, new technology approaches need to be developed in order to cope with certain challenges that last mile delivery entails, such as navigation in an urban environment. Autonomous delivery robots can help overcome these challenges. We propose a method for performing mixed reality (MR) simulation with ROS-based robots using Unity, which synchronizes the real and virtual environment, and simultaneously uses the sensor information of the real robots to locate themselves and project them into the virtual environment, so that they can use their virtual doppelganger to perceive the virtual world. Using this method, real and virtual robots can perceive each other and the environment in which the other party is located, thereby enabling the exchange of information between virtual and real objects. Through this approach a more realistic and reliable simulation can be obtained. Results of the demonstrated use-cases verified the feasibility and efficiency as well as the stability of implementing MR using Unity for Robot Operating System (ROS)-based robots.

I. INTRODUCTION

Advanced applications that rely on Intelligent Transportation Systems (ITS) combine Information and Communication Technologies (ICT) in a connected environment in which sensors acquire information relevant for services related to different modes of transport, contributing to their more efficient and sustainable use. Before testing the systems in a real world environment, it is essential that the system performance is validated in a controlled environment using simulation frameworks.

In the context of transport logistics, the growth of e-commerce has increased customer demand and new technology approaches need to be developed to cope with the challenges occurring during the final step of the delivery process from a facility to the end user (last mile). These challenges refer, for example, to navigation in an urban environment, including parking and its effect on fuel consumption, fuel costs and CO₂ emissions. Autonomous delivery robots can help overcome these challenges.

We present a method for implementing mixed reality applications for Robot Operating System [1] ROS-based mobile

robots in the open source game engine Unity 3D [2], thus extending the work presented in [3]. Unity 3D has been widely used in the development of ITS by connecting it to advanced 3D modeling software for urban environments including CityEngine [4], Simulation of Urban MObility (SUMO) [5], and ROS [6], [7], [8], [9], [10].

The connection of Unity and ROS can greatly simplify the development and simulation process of autonomous delivery robots.

In this work, we contribute to the field of multirobot simulation by combining a simulated world with the physical world. This approach makes it possible to use a small number of real robots in combination with a large number of virtual robots to perform a multi-robot simulation in a real environment and therefore obtain more realistic and reliable simulation results while improving the efficacy of the entire system.

This proposed visualization of robot models and data, also described as the integration of physical objects into a virtual environment (Augmented Virtuality (AV)), can be seen as a special instance of Mixed Reality (MR) [11]. According to [12] MR has to be able to spatially map physical and virtual objects alongside one another in real-time in a fusion of the physical world and the virtual world to create new environments and visualizations in which physical and virtual objects coexist and interact in real-time. Therefore, our work may help lay the foundation for the development of MR applications for mobile delivery robots in ITS.

The remainder of this paper is organized as follows: Section II introduces the previous works in the field. The methodology to use Unity to create mixed reality applications for ROS-based robots is presented in Section III. Afterwards, the validation experiments are evaluated in Section IV. Finally, Section V concludes the paper and presents possible future work.

II. RELATED WORK

Before implementing MR applications for ROS-based robots in Unity, it is first necessary to establish reliable communication between ROS and Unity. Several libraries have been written to integrate Unity with ROS. For example, the *rosbridgelib* uses *rosbridge* [13] for communication and defines messages in Unity to establish bidirectional communication at a later point [14]. The approach presented in this paper adheres to this library and added several scripts to extend its support for different ROS message types.

Johannes Kepler University Linz; Chair Sustainable Transport Logistics 4.0, Altenberger Straße 69, 4040 Linz, Austria. {yuzhou.liu, georg.novotny, nikita.smirnov, walter.morales.alvarez, cristina.olaverri-monreal}@jku.at

² UAS Technikum Wien, Hoechsttaedtplatz 6, 1200 Vienna, Austria

³ Ural Federal University, Department of Communications Technology, Ekaterinburg, Russia

It is worth mentioning that Unity is used as a full virtual simulation tool rather than a mixed reality platform, examples of the latter being in the modeling and simulation of the humanoid robot in [15] and the unmanned aerial vehicle (UAV) project introduced in [16].

As demand changes and mixed reality technologies mature, applications similar to ours in this area are expected to increase. Related research that laid the foundation for our work is for example the implementation of a ROS-based robot controlled by Unity and VR devices [17]. The authors successfully imported ROS map information into Unity and created a new scene accordingly, which also enabled the visualization of the robot in Unity to some extent. The paper described the method for sending messages to control the robot through Unity, but the acquisition and visualization of laser scan data in Unity was not mentioned, so that the effect of the process on the final results could not be verified.

In order to solve the problem of matching the robot's localization and Unity scene, we refer to the work in [18]. The authors also connected Unity and ROS and accomplished the synchronization of real and virtual robot, but Unity was only used as a control medium to transfer commands from the gaming device to ROS.

A similar method was used in [12] to create a mixed reality implementation for multiple UAVs based on ROS and Unity. The UAVs' positions were tracked through an additional Virtual-Reality Peripheral Network (VRPN) protocol, which is a device-independent and network-transparent system designed specifically for VR peripherals [19]. ROS was only used to control UAVs, but not connected with Unity.

In a further work [20] the cooperative Simultaneous Localization and Mapping (SLAM) and map merging methods were used so that 3 robots could derive their relative position and orientation in the same environment without knowing their initial position. This implementation solved real-time self-localization issues.

Another work [21] provided a more detailed discussion of similar research, further validating the feasibility of the MR approach and providing a more complete and accurate mathematical framework. Furthermore, in [22] and [20] the authors adopted cooperative SLAM and demonstrated the detection performance of the poses of each robot in the group. In the last work the authors additionally used ROS to share robot information related to position, rotation and scale.

In the work presented here we have successfully implemented mixed reality applications using only Unity and ROS and a smart mobile device. At the same time, we successfully enabled the real robot's ability to perceive both the virtual environment and other real-world and virtual robots.

III. MATCHING VIRTUAL AND PHYSICAL WORLDS (MIXED REALITY)

This section explains in detail the process of matching the virtual and physical worlds using ROS and Unity in two stages.

- 1) In the first stage, Unity3D is used to visualize and project the robot models from the virtual environment onto the real world.
- 2) Afterwards, in a second step, we use ROS to share the sensor data and project the current status of the real robots to their virtual doppelgangers in the virtual environment.

A. Visualization in Unity

We explain next the concepts required to apply the proposed method.

1) *AR Foundation*: We relied on the Unity 3D AR foundation package for implementing augmented reality functions. The package supports devices such as smart phones or tablets. We mainly used its planar surface detection function to overlay the virtual scene on the detected plane and make it match the physical surroundings. Its world tracking function made it possible to keep all the information related to the *transform* component that is part of each object within the Unity Scene, and this function is then used to store and manipulate the position, rotation and scale of the object [2].

2) *Modeling and Coordinate Systems*: Although the AR foundation can provide both horizontal and vertical plane detection and even simple spatial mapping, in order to obtain better performance, it is still required to model the environment manually.

Since we used robots that were equipped with 2D Lidars, we relied on the method in [17] that used a scanned map of the robots for modeling the environment. But instead of just using point clouds to represent the map we extended the approach to interact with the virtual scene by using models as game objects to rebuild the environment with collider components that rely on Unity's built-in collision system.

By importing the map and using the polygon building tool provided by Unity, we can quickly obtain a rough model of the desired environment, and then manually adjust to get the final, more accurate model.

For AR applications, the scaling of the models is very important. In theory, one unit length in Unity is equivalent to one meter in reality. We adjusted our model to a scale that matches reality by measuring and calculating proportions. In Figure 2a a properly scaled robot model in comparison with the size of a real robot is depicted.

Finally, we matched the coordinate systems in Unity and ROS. As Unity's game objects adopt a hierarchy structure, by arranging the hierarchies, we were able to obtain a local coordinate system that was not affected by scale change and that coincided with the ROS zero point.

3) *Installing to Device and Testing*: Because the sensors on the mobile device are needed to complete the AR function, we implemented an application (APP) and installed it on an Android smart phone. When the APP started running, it was necessary to move the camera of the mobile device to complete the plane detection of the entire environment. At this time, Unity was not connected to ROS.

Next, according to the previously written scripts, the experimental scene was initialized by tapping the screen. The manipulations press and drag with fingers could be used to fine-tune the *transform* of the environment. The connection between Unity and ROS was then established at the same time as the experimental scenario was initialized. The proposed communication flow is depicted in Figure 1. The effect

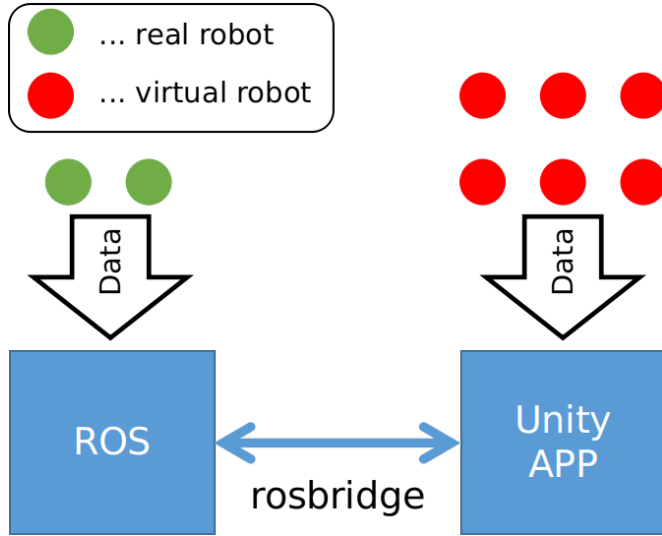


Fig. 1. Overview of the communication flow

of matching the virtual scene with the real world, as well as the proportional relationship between the virtual robot model and the environment are visualized in Figure 2b.

B. Self-Localization and Data Sharing within ROS

As previously mentioned we used ROS to share the robot *transform* information. To this end we adhered to the *amcl* package, which is used as a probabilistic localization system for robots moving in 2D [23], so that their location and orientation can be recorded to be later shared within the ROS *tf* system. Since in our approach the virtual and real scenario are completely matched with the map, the virtual and real robots can obtain the relative positions of each other. In fact, after having synchronized the coordinate systems, a single robot can also be accurately mapped into the virtual space. By combining the AR foundation approach with the described method, we can then visualize the position of a real robot in the virtual environment and in the AR scene. Figure 3a illustrates how a real robot is projected into a virtual space, the virtual environment and objects can then be embedded in the real world using mobile devices. After correctly projecting a real robot into a virtual space, its doppelganger can be used to emit another set of laser scans. By sending the scanned data back to ROS the real robot can then perceive virtual objects. The final AR view is depicted in Figure 3b. As shown in the upper left corner of the picture, instead of using the virtual robot model, we used an additional translucent model as the doppelganger of the real robot for better visualization purposes.

IV. SYSTEM EVALUATION AND RESULTS

To evaluate the implementation method, we designed two use-cases as described in this section.

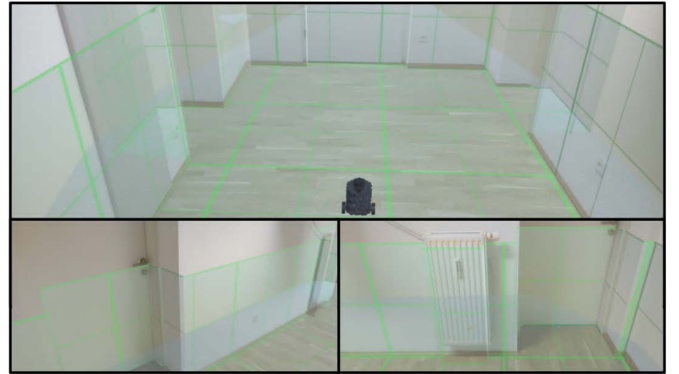
A. Robot Following Behavior

We developed an algorithm to test the feasibility of our approach by implementing a scenario in which a leading real robot (leader robot) was followed by a virtual robot (virtual follower robot) that in turn was followed by a real robot (real follower robot) as Figure 4 illustrates.

The leader robot was required to perform self-localization, thereby projecting its position into a virtual environment and generating a doppelganger. The virtual follower robot was represented in the Unity environment, so that it could directly obtain its own *transform* information and then transfer it to ROS for sharing. In order for this second robot to perceive the leader doppelganger in the virtual environment we implemented a virtual laser scan. The real follower robot in the third position was then required to perform self-localization by scanning and measuring the environment, to be projected into the virtual space. Then using the laser scan information of the doppelganger in the virtual environment, the real follower robot was able to perceive the virtual robot in front and then follow its movement.

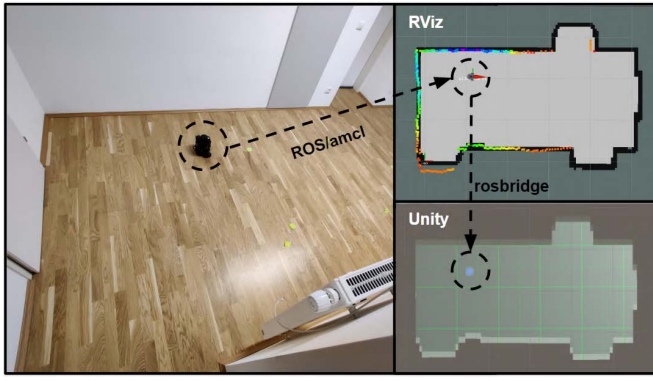


(a) Properly scaled virtual robot model (left) located on the detected plane and compared with the real robot (right)

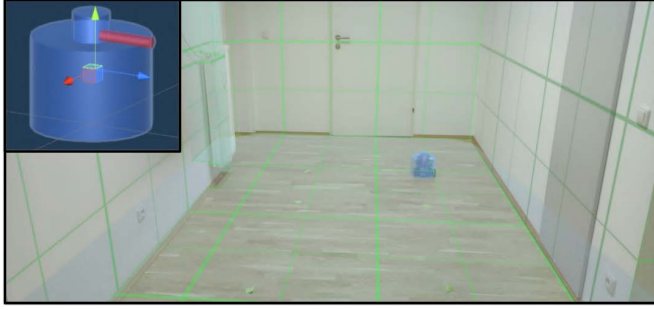


(b) Illustration of matching the virtual scene with the real world, as well as the proportional relationship between the virtual robot model and the environment

Fig. 2. Overview of the virtual robot inside the virtual and real environment



(a) Projection of a real robot into a virtual space



(b) Augmented reality view resulting from the proposed approach

Fig. 3. Visualization of a real robot in a virtual environment as a result of the matching of scaling and localization

Through this use case, we successfully verify the feasibility of using Unity to perform MR simulation for ROS-based robots. The transform information of the real robot is accurately projected into the virtual space. Mutual awareness between real and virtual robots has also been successfully established.

In this use case, each robot's actions are determined by its own program, rather than an overall control algorithm, which means that the robots' transform information is not fully shared between every robot, and each robot can only sense the target in front of it based on its own sensors.

In many cases in the real world, robots are controlled by a central system. In order to verify the performance of this implementation for this central system, we introduce a second use-case in the next subsection.

B. Multi-robot Roundabout Entry and Exit Behavior

To test the performance of a multi-robot hybrid simulation we developed a use case with eight Turtlebot3 [24], two of which were real and six virtual. Each robot obtained its pose through ego localization and shared it with the others. The selected test scenario consisted of two roundabouts with one lane each that were connected by a two-lane road (one lane for each direction), as visualized in Figure 5.

Traffic was generated as follows: 4 robots were placed on each roundabout forming a traffic queue. Each robot was initialized with a unique speed and controlled to follow the lane of the roundabout using a PID controller. Then a manual

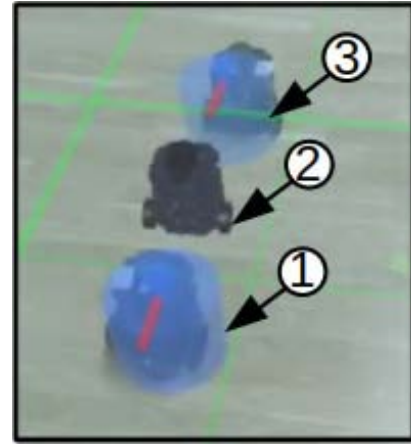


Fig. 4. Illustration of the performed robot-following behavior experiment

1) leader robot; 2) real follower robot; 3) virtual follower robot

request was sent to a robot to join the other traffic queue. To do so, they needed to leave the current queue, drive along the two-lane road and make a safe turn onto the new roundabout. Notice that this scenario was designed that only one robot from one queue can leave per cycle.

Since the robots exhibit different speeds, two algorithms were used to control the current velocity. To avoid collisions while being on the roundabout, a basic linear control concept was applied to alter the velocity of a robot based on the distance acquired from the simulated Light Detection And Ranging (LIDAR) sensors inside of Unity. Traffic at the entrance and exit of the roundabout, on the other hand, was managed by an autonomous intersection manager (IM). Notice that each roundabout has its own IM to handle the requests at their intersection. While robots are inside the IM area they send their current velocity and type of movement (exit intersection, follow traffic queue, enter intersection) to the IM which then handles the traffic as described in algorithm 1.

Algorithm 1: Intersection Manager

```

input : velocity, movement
output: velocity

queue ← new list
for  $i \leftarrow 0$  to # robots in IM area do
    queue.append(i)
while True do
    for  $i \leftarrow 0$  to # queue - 1 do
        pose = get_pose_of_robots(queue[i])
        time = calculate_time_to_intersection(pose)
        if possible_collision(time, queue[i+1]) then
            adjust_velocity_of_robot(i+1)
        else
            return velocity
    queue.remove(queue[i])

```

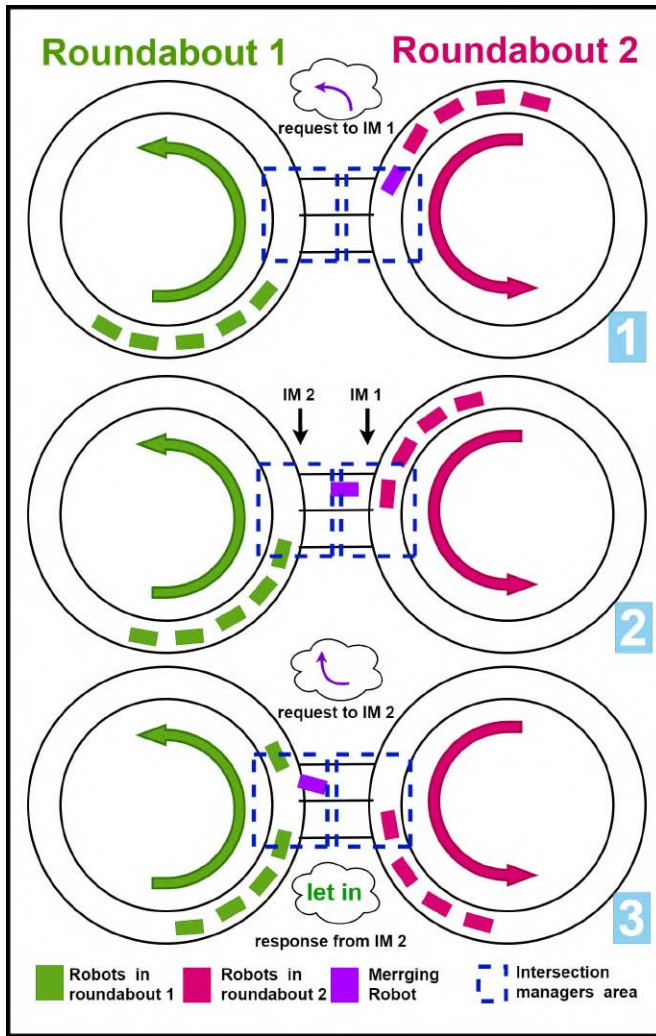


Fig. 5. Scenario description; 1: merging robot sends a request to leave from roundabout 2; 2: merging robot enters into an intersection queue; 3: merging robot sends a request to join roundabout 1 and after that robots from roundabout 1 get a message to let in the merging robot

After receiving a request, the manager adds the robot's request to a queue and handles the incoming robot requests based on the FIFO principle. The current pose and velocity of each first robot is used to calculate the time to intersection (TTI), utilizing a differential drive kinematic motion model, and calculates possible conflicts for the following robots in the queue based on this. If a potential conflict is detected (i.e. collision), the manager adjusts the speed of the robot whose request arrived after the current first robot. In case of no conflict, the value returned by the manager is the original speed of the robot. Finally the IM removes the current request from the queue and starts over. Figure 6 visualizes the test scenario. As it can be seen when comparing the left and right images the IM updated the current velocity of the real robot that was marked within a circle in the figure, so that it would not crash into the simulated robot in the roundabout.

Through a more sophisticated simulation, we verified

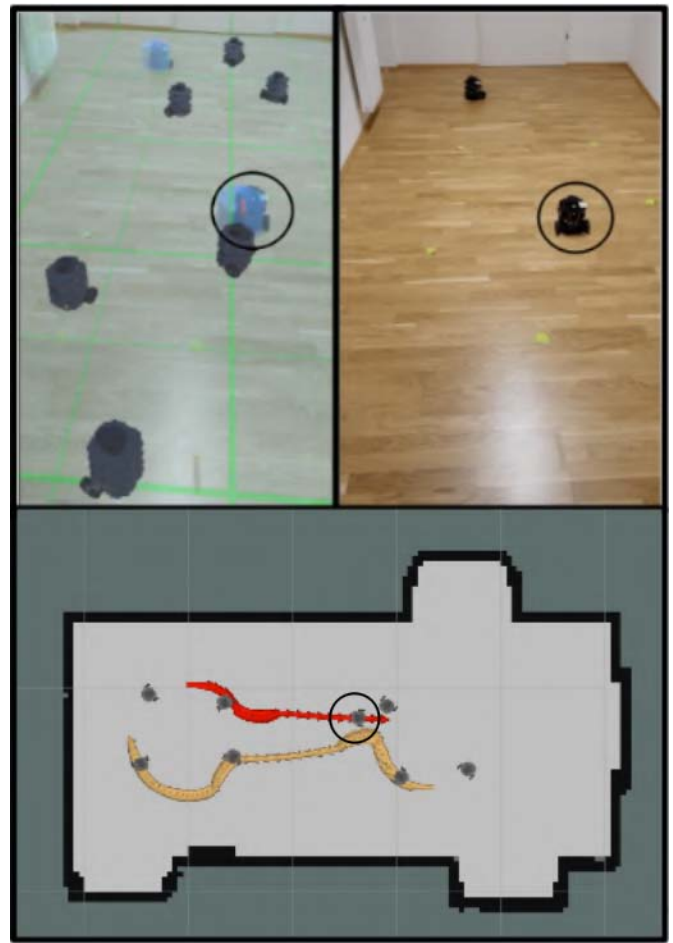


Fig. 6. Illustration of the experiment performed, in which a robot waits to be joined in other roundabout by IM; Left: mixed reality view with virtual and real robots (in blue); Right: real world view in which the virtual robots are not visible; Bottom: visualization inside rviz, trajectories of merging robots are colored in red and orange

the stability of the approach. Whether virtual or real, the position and orientation information of each robot can be accurately updated in real-time, and the mutual perception can help them avoiding collisions with each other. Meanwhile, this approach of combining a simulated world with the physical world makes it possible to perform a large-scale simulation that integrates small-scale field tests, thus obtaining more realistic and reliable simulation results than in pure virtual environments and improving the efficacy of the entire system.

V. CONCLUSION AND FUTURE WORK

The work we presented describes a method relying on mixed reality to be applied in the testing of delivery robots behavior and interaction in a realistic simulation framework. Two concrete simulation examples were described, in which the visualization was managed by Unity 3D and the data shared by the robots was managed by the self localization

algorithm that was implemented in ROS.

The results from the use cases showed the feasibility and efficiency of the Unity-based MR simulation for ROS-based robots. The matching of virtual and real environments was established, the mutual perception of virtual and real robots was successfully achieved as they were following each other. By using multiple virtual and real robots to simulate complex behaviors, we also successfully verified the stability of the proposed system. At the same time, due to the powerful extend-ability of Unity itself, integrating designed MR use cases into other large-scale simulations is also possible.

Future work will aim at performing spatial mapping dynamically to improve the universality of the MR applications and integrating the results into the “autonomous vehicles” option of the 3DCoAutoSim simulation framework [7].

ACKNOWLEDGMENT

This work was supported by the Austrian Ministry for Climate Action, Environment, Energy, Mobility, Innovation and Technology (BMK) Endowed Professorship for Sustainable Transport Logistics 4.0.

REFERENCES

- [1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “Ros: an open-source robot operating system,” in *ICRA workshop on open source software*, vol. 3, no. 3.2. Kobe, Japan, 2009, p. 5.
- [2] D. Helgason, “Unity 1.0 is shipping,” [Last Accessed: 2019-03-31]. [Online]. Available: <https://forum.unity.com/threads/unity-1-0-is-shipping.56/>
- [3] A. Hussein, F. Garcia, and C. Olaverri-Monreal, “Ros and unity based framework for intelligent vehicles control and simulation,” in *2018 IEEE International Conference on Vehicular Electronics and Safety (ICVES)*. IEEE, 2018, pp. 1–6.
- [4] “ESRI City Engine,” [Last Accessed: 2019-05-19]. [Online]. Available: <https://www.esri.com/en-us/arcgis/products/esri-cityengine/overview>
- [5] SUMO, “DLR - Institute of Transportation Systems - SUMO – Simulation of Urban MObility,” sumo.dlr.de, Accessed on: 20-Jul-2016.
- [6] C. Biurrún, L. Serrano-Arriazu, and C. Olaverri-Monreal, “Microscopic driver-centric simulator: Linking unity3d and sumo,” in *World Conference on Information Systems and Technologies*. Springer, 2017, pp. 851–860.
- [7] A. Hussein, A. Diaz-Alvarez, J. M. Armingol, and C. Olaverri-Monreal, “3DCoAutoSim: Simulator for Cooperative ADAS and automated vehicles,” in *IEEE International Conference on Intelligent Transportation Systems (ITSC2018)*. IEEE, 2018, pp. 1–6.
- [8] L. Artal-Villa and C. Olaverri-Monreal, “Vehicle-Pedestrian Interaction in SUMO and Unity3D,” in *Proceedings WorldCIST 2019*. Springer International Publishing AG, Cham, Switzerland, 2019, p. .
- [9] C. Olaverri-Monreal, J. Errea-Moreno, and A. Diaz-Alvarez, “Implementation and evaluation of a traffic light assistance system in a simulation framework based on v2i communication,” *Journal of Advanced Transportation*, 2018.
- [10] C. Olaverri-Monreal, J. Errea-Moreno, A. Díaz-Álvarez, C. Biurrún-Quel, L. Serrano-Arriazu, and M. Kuba, “Connection of the SUMO Microscopic Traffic Simulator and the Unity 3D Game Engine to Evaluate V2X Communication-Based Systems,” *Sensors*, vol. 18, no. 12, p. 4399, 2018.
- [11] P. Milgram and F. Kishino, “A taxonomy of mixed reality visual displays,” *IEICE TRANSACTIONS on Information and Systems*, vol. 77, no. 12, pp. 1321–1329, 1994.
- [12] W. Hoenig, C. Milanes, L. Scaria, T. Phan, M. Bolas, and N. Ayanian, “Mixed reality for robotics,” in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2015, pp. 5382–5387.
- [13] R. T. Jonathan Mace and J. Lee, “Ros bridge suite,” [Last Accessed: 2019-03-31]. [Online]. Available: http://wiki.ros.org/rosbridge_suite
- [14] M. C. Thorstensen, “Ros bridge lib,” [Last Accessed: 2019-03-31]. [Online]. Available: <https://github.com/MathiasCiarlo/ROSBridgeLib>
- [15] W. A. Mattingly, D.-j. Chang, R. Paris, N. Smith, J. Blevins, and M. Ouyang, “Robot design using unity for computer games and robotic simulations,” in *2012 17th International Conference on Computer Games (CGAMES)*. IEEE, 2012, pp. 56–59.
- [16] Y. Hu and W. Meng, “Rosunitysim: Development and experimentation of a real-time simulator for multi-unmanned aerial vehicle local planning,” *Simulation*, vol. 92, no. 10, pp. 931–944, 2016.
- [17] R. Codd-Downey, P. M. Forooshani, A. Speers, H. Wang, and M. Jenkin, “From ros to unity: Leveraging robot and virtual environment middleware for immersive teleoperation,” in *IEEE International Conference on Information and Automation (ICIA)*. IEEE, 2014, pp. 932–936.
- [18] J. J. Roldán, E. Peña-Tapia, D. Garzón-Ramos, J. de León, M. Garzón, J. del Cerro, and A. Barrientos, “Multi-robot systems, virtual reality and ros: Developing a new generation of operator interfaces,” in *Robot Operating System (ROS)*. Springer, 2019, pp. 29–64.
- [19] R. M. Taylor II, T. C. Hudson, A. Seeger, H. Weber, J. Juliano, and A. T. Helser, “Vrpn: a device-independent, network-transparent vr peripheral system,” in *Proceedings of the ACM symposium on Virtual reality software and technology*. ACM, 2001, pp. 55–61.
- [20] S. Bultmann, L. Matignon, and O. Simonin, “Multi-robot navigation and cooperative mapping in a circular topology,” Ph.D. dissertation, INSA Lyon, 2017.
- [21] A. K. Nasir, “Cooperative simultaneous localization and mapping framework,” 2014.
- [22] M. Walter and J. Leonard, “An experimental investigation of cooperative slam,” *IFAC Proceedings Volumes*, vol. 37, no. 8, pp. 880–885, 2004.
- [23] B. P. Gerkey, “amcl,” [Last Accessed: 2019-06-27]. [Online]. Available: <http://wiki.ros.org/amcl>
- [24] R. Amsters and P. Slaets, “Turtlebot 3 as a robotics education platform,” in *Robotics in Education*, M. Merdan, W. Lepuschitz, G. Koppensteiner, R. Balogh, and D. Obdržálek, Eds. Cham: Springer International Publishing, 2020, pp. 170–181.